

Angular Signals – Complete Interview Preparation Guide

Beginner to Advanced

Angular 17–21+

1. What is a Signal in Angular?

Interview answer:

A Signal is a reactive wrapper around a value that notifies Angular when the value changes. Angular uses signals to track where state is read and update the relevant parts of the application efficiently.

Signals were introduced in Angular 16 and are commonly used for managing component state.

Key features:

- Reactive state management.
- Fine-grained dependency tracking.
- Automatic dependency tracking for computed signals.
- Supports primitive values, objects and arrays.
- Works with Angular's change detection, including zoneless applications.

Simple example

TypeScript

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <h2>Count: {{ count() }}</h2>

    <button (click)="increment()">Increment</button>
  `
})
export class CounterComponent {

  count = signal(0);

  increment() {
    this.count.update(value => value + 1);
  }
}
```

```
}  
}
```

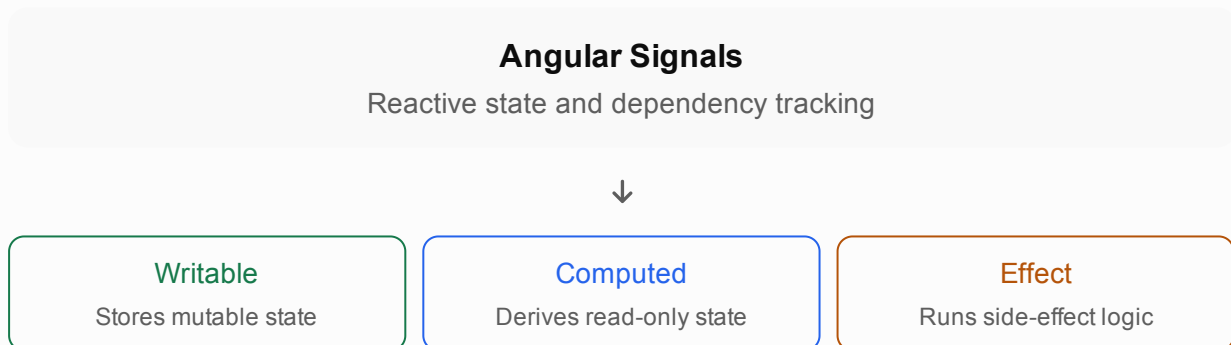
Explanation:

1. `signal(0)` creates a signal with an initial value of 0.
2. `count()` reads the current value.
3. `update()` calculates and sets a new value.
4. Angular marks the component for updating when a signal read in its template changes.

2. Types of Signals in Angular

There are three fundamental signal concepts commonly discussed in interviews.

Angular Signal Types



Technically, `effect()` is a reactive API rather than a Signal type. Angular also provides `linkedSignal()`, `input()`, `model()` and resource-related APIs.

Type	Purpose	Writable?
<code>signal()</code>	Stores reactive state	Yes
<code>computed()</code>	Calculates derived values	No
<code>effect()</code>	Executes side effects	Not a signal
<code>linkedSignal()</code>	Writable state linked to another reactive source	Yes
<code>input()</code>	Receives input from a parent	No
<code>model()</code>	Supports two-way component binding	Yes

3. Writable Signal – `signal()`

A writable signal stores a value that can be modified using `set()` or `update()`.

Syntax

TypeScript

```
const count = signal(0);
```

Angular infers the signal's type from its initial value.

TypeScript

```
const count = signal<number>(0);  
const name = signal<string>('Venkat');  
const isActive = signal<boolean>(true);
```

Important methods

Method	Purpose
<code>signal()</code>	Creates a writable signal
<code>set()</code>	Replaces the current value
<code>update()</code>	Updates based on the previous value
<code>asReadonly()</code>	Exposes a read-only signal
<code>()</code>	Reads the current value

Example: set() and update()

TypeScript

```
import { signal } from '@angular/core';  
  
const count = signal(10);  
  
// Read  
console.log(count()); // 10  
  
// Set new value  
count.set(20);  
  
console.log(count()); // 20  
  
// Update existing value
```

```
count.update(value => value + 5);

console.log(count()); // 25
```

Difference between set() and update()

set()

Replaces the value

Accepts a new value

```
count.set(10)
```

update()

Calculates from the current value

Accepts an updater function

```
count.update(x => x + 1)
```

Interview tip: Use `set()` when the new value is already known. Use `update()` when the new value depends on the existing value.

4. Computed Signal – computed()

Interview answer:

A computed signal is a read-only signal that derives its value from other signals. It automatically tracks dependencies, recalculates when necessary and caches its result.

Example: Calculate total price

TypeScript

```
import { signal, computed } from '@angular/core';

export class ProductComponent {

  price = signal(100);
  quantity = signal(2);

  total = computed(() =>
    this.price() * this.quantity()
  );

  changeQuantity() {
    this.quantity.set(5);
  }
}
```

HTML:

HTML

```
<p>Price: {{ price() }}</p>
<p>Quantity: {{ quantity() }}</p>
<p>Total: {{ total() }}</p>

<button (click)="changeQuantity()">
  Change Quantity
</button>
```

Output:

Initially:

```
Price: 100
Quantity: 2
Total: 200
```

After clicking:

```
Price: 100
Quantity: 5
Total: 500
```

Important characteristics

- Computed signals are read-only.
- Values are calculated lazily, on demand.
- Results are memoized (cached).
- Dependencies are tracked dynamically.
- A computed signal recalculates when read after a dependency change invalidates its cached value.

You cannot write:

TypeScript

```
this.total.set(500); // Error
```

Interview trap: A computed signal is not recalculated immediately every time a dependency changes. It is invalidated and recalculated when its value is next requested.

5. Effect – effect()

Interview answer:

An effect is a reactive function that runs when its tracked signal dependencies change. It is mainly used to synchronize signal state with external systems.

Examples include logging, browser storage, third-party charts and external APIs.

Example

TypeScript

```
import {
  Component,
  signal,
  effect
} from '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  template: `
    <p>{{ count() }}</p>

    <button (click)="increment()">
      Increment
    </button>
  `
})
export class CounterComponent {

  count = signal(0);

  constructor() {

    effect(() => {
      console.log('Count changed:', this.count());
    });

  }

  increment() {
    this.count.update(value => value + 1);
  }
}
```

Console output after initial execution and successive clicks:

```
Count changed: 0
Count changed: 1
Count changed: 2
```

Effects execute asynchronously during Angular's synchronization process. Multiple synchronous updates may be coalesced, so an effect is not guaranteed to run once for every intermediate value.

When should we use effect()?

Use case	Recommended?
Logging signal changes	Yes
Synchronizing local storage	Yes
Updating third-party chart	Yes
Calculating total price	No, use <code>computed()</code>
Copying one signal into another	Generally avoid
Fetching data	Prefer dedicated async APIs or RxJS patterns

Important: Avoid using effects to propagate ordinary application state. This can create unnecessary updates or circular dependencies.

6. Comparison: signal() vs computed() vs effect()

Feature	signal()	computed()	effect()
Purpose	Store state	Derive state	Side effects
Writable	Yes	No	N/A
Returns	WritableSignal	Signal	EffectRef
Lazy	N/A	Yes	No
Cached value	Stores state	Yes	No
Tracks dependencies	No derivation	Yes	Yes
Example	Counter	Total price	Logging

Interview answer in one line:

`signal()` stores data, `computed()` derives data, and `effect()` reacts to changes by performing side effects.

7. `linkedSignal()` – Writable Derived State

A linked signal creates writable state that can be recalculated when a reactive source changes.

It is useful when you want to derive an initial value from another signal but still allow manual changes.

Example: Product selection

TypeScript

```
import {
  signal,
  linkedSignal
} from '@angular/core';

export class ProductComponent {

  products = signal([
    'Laptop',
    'Mobile',
    'Tablet'
  ]);

  selectedProduct = linkedSignal(() =>
    this.products()[0]
  );

  selectProduct() {
    this.selectedProduct.set('Mobile');
  }

  changeProducts() {
    this.products.set([
      'Keyboard',
      'Mouse'
    ]);
  }
}
```

Behavior:

Action	Selected product
Initial	Laptop
<code>selectProduct()</code>	Mobile
<code>changeProducts()</code>	Keyboard

When the products signal changes, the linked signal recalculates its value.

computed() vs linkedSignal()

computed()	linkedSignal()
Read-only	Writable
Derived value	Derived and manually editable
Cannot call <code>set()</code>	Supports <code>set()</code>
Total price	Selected product

Interview tip: Use `linkedSignal()` for state such as a selected item that should reset or be reconciled when its source collection changes.

8. Input Signal – input()

Input signals allow child components to receive reactive values from parent components.

Unlike traditional `@Input()`, the value is accessed using signal syntax.

Child component

TypeScript

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `
    <h3>{{ username() }}</h3>
  `,
})
export class ChildComponent {

  username = input<string>('');
```

```
}
```

Parent component

TypeScript

```
username = signal('Venkat');
```

HTML

```
<app-child [username]="username()" />
```

The child receives the value reactively.

Required input

TypeScript

```
username = input.required<string>();
```

The parent must provide the required input.

Important: Input signals are read-only within the child.

TypeScript

```
this.username.set('Ram'); // Error
```

9. Model Signal – model()

`model()` creates a writable input that supports two-way binding between parent and child components.

Child component

TypeScript

```
import { Component, model } from '@angular/core';

@Component({
  selector: 'app-counter',
```

```

standalone: true,
template: `
  <p>{{ count() }}</p>

  <button (click)="increment()">
    Increment
  </button>
`
}))
export class CounterComponent {

  count = model(0);

  increment() {
    this.count.update(x => x + 1);
  }
}

```

Parent component

TypeScript

```
count = signal(10);
```

HTML

```

<app-counter [(count)]="count" />

<p>Parent Count: {{ count() }}</p>

```

When the child increments its count, the parent signal is updated through two-way binding.

input() vs model()

input()

One-way binding

Read-only

Parent to child

```
input(0)
```

model()

Two-way binding

Writable

Parent and child synchronization

```
model(0)
```

10. Signal with Objects and Arrays

Signals support complex data types, but changes should generally be made immutably.

Array example

TypeScript

```
import { signal } from '@angular/core';

export class EmployeeComponent {

  employees = signal<string[]>([
    'Ram',
    'Venkat'
  ]);

  addEmployee() {

    this.employees.update(list => [
      ...list,
      'Kumar'
    ]);

  }

  removeEmployee() {

    this.employees.update(list =>
      list.filter(name => name !== 'Ram')
    );

  }
}
```

Avoid:

TypeScript

```
this.employees().push('Kumar');
```

This mutates the existing array without notifying the signal of a new value.

Correct:

TypeScript

```
this.employees.update(list => [
  ...list,
```

```
'Kumar'  
]);
```

Object example

TypeScript

```
employee = signal({  
  id: 1,  
  name: 'Venkat'  
});  
  
updateName() {  
  this.employee.update(emp => ({  
    ...emp,  
    name: 'Ram'  
  }));  
}
```

Interview trap: Signals do not deeply observe object properties. Their default equality comparison uses `Object.is()`, so updating with a new object or array reference is the usual approach.

11. Signal Dependency Tracking

Angular automatically tracks signals read inside `computed()` and `effect()`.

Dependencies are dynamic: only signals actually read during the latest execution are tracked.

Example

TypeScript

```
import { signal, computed } from '@angular/core';  
  
showCount = signal(false);  
  
count = signal(10);  
  
message = computed(() => {  
  
  if (this.showCount()) {  
    return `Count: ${this.count()}`;  
  }  
  
  return 'Count hidden';  
});
```

```
});
```

When `showCount()` is false, the computed signal does not read `count()`.

Therefore, changes to `count` do not invalidate this computed value until the branch that reads it becomes active.

This is called **dynamic dependency tracking**.

12. Signal vs Observable

One of the most frequently asked Angular interview questions.

Signal	Observable
Represents current reactive state	Represents a stream of events or values
Read synchronously using <code>()</code>	Subscribe or use async pipe
Always has a current value	May have no initial value
No subscription required to read	Subscription generally required
Best for UI state	Best for async streams and events
Built into Angular	Provided by RxJS

Example: Observable

TypeScript

```
count$ = interval(1000);
```

HTML

```
<p>{{ count$ | async }}</p>
```

Example: Signal

TypeScript

```
count = signal(10);
```

HTML

```
<p>{{ count() }}</p>
```

Interview answer: Signals are useful for synchronous state management and derived UI state. Observables are useful for asynchronous streams, HTTP workflows, cancellation, debouncing and complex event composition.

Angular provides interoperability utilities:

TypeScript

```
toSignal()  
toObservable()
```

`toSignal()` converts an Observable into a Signal, while `toObservable()` exposes signal values as an Observable.

13. Signals and Change Detection

When Angular reads a signal inside a component template, it tracks the signal as a dependency of that component.

When the signal changes, Angular marks the component for checking.

This works with both Default and OnPush change detection strategies.

Example

TypeScript

```
@Component({  
  selector: 'app-counter',  
  changeDetection: ChangeDetectionStrategy.OnPush,  
  template: `  
    <p>{{ count() }}</p>  
  `,  
})  
export class CounterComponent {  
  
  count = signal(0);  
  
}
```

Even with `OnPush`, Angular tracks the signal read in the template and marks the component when the value changes.

Important distinction: Signals do not automatically make every component update immediately or bypass change detection entirely. They help Angular identify reactive dependencies and schedule or perform the appropriate synchronization.

14. Signal Equality Function

Angular signals use `Object.is()` equality by default.

You can provide a custom equality function.

TypeScript

```
const count = signal(0, {  
  equal: (a, b) => a === b  
});
```

If Angular considers the old and new values equal, it does not notify dependent consumers of a change.

Custom equality is useful in specific cases, but expensive deep comparisons should be used carefully.

15. untracked()

`untracked()` allows you to read a signal without registering it as a dependency of the current reactive computation.

Example

TypeScript

```
import {  
  signal,  
  effect,  
  untracked  
} from '@angular/core';  
  
count = signal(0);  
username = signal('Venkat');
```

```

constructor() {

  effect(() => {

    console.log(this.count());

    console.log(
      untracked(() => this.username())
    );

  });

}

```

Here:

- Changing `count` triggers the effect.
- Changing only `username` does not trigger the effect.

Interview tip: Use `untracked()` when you need to read a signal incidentally without making it a reactive dependency.

16. asReadOnly()

You can expose a writable signal as a read-only signal.

This is particularly useful in services.

TypeScript

```

import { Injectable, signal } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CounterService {

  private readonly _count = signal(0);

  readonly count = this._count.asReadOnly();

  increment() {
    this._count.update(x => x + 1);
  }

}

```

A component can read:

TypeScript

```
this.counterService.count();
```

But cannot call:

TypeScript

```
this.counterService.count.set(10);
```

This supports encapsulation and controlled state updates.

Note that `asReadonly()` does not deeply freeze objects stored inside signals.

17. Resource APIs for Asynchronous State

Angular also provides resource-related APIs for asynchronous reactive state.

Examples include `resource()` and `httpResource()`.

These APIs can expose state such as:

- Loading
- Resolved value
- Error
- Request status

They are useful for asynchronous data that depends on reactive inputs.

For example, a resource can reload data when a selected user ID changes.

Interview distinction: A resource is not simply another writable signal. It coordinates asynchronous operations and exposes reactive state. API stability and availability can vary by Angular version.

18. Frequently Asked Angular Signal Interview Questions

15 interview questions · Tap to reveal answers

Expand all

1. What is an Angular Signal?

A reactive wrapper around a value that allows Angular to track reads and notify consumers when the value changes.

2. What are the three main signal concepts?

Writable signals, computed signals and effects. Technically, an effect is a reactive API rather than a signal type.

3. What is the difference between `set()` and `update()`?

`set()` replaces the current value; `update()` calculates a new value from the current value.

4. Can we modify a computed signal?

No. A computed signal is read-only and derives its value from other signals.

5. Are computed signals lazy?

Yes. Their derivation is evaluated when first read and recalculated on demand after dependencies invalidate the cached value.

6. Can we use signals with `OnPush`?

Yes. When a signal read in an `OnPush` component template changes, Angular marks that component for checking.

7. Does mutating an array trigger signal updates?

Not by itself. Use `set()` or `update()` with a new array reference for reliable notification.

8. What is the difference between `computed()` and `effect()`?

`computed()` produces cached derived state. `effect()` performs side effects when tracked dependencies change.

9. What is `linkedSignal()`?



It creates writable state linked to a reactive source and recalculates it when that source changes.

10. What is the difference between `input()` and `model()`?



`input()` creates a read-only signal input. `model()` creates a writable input that supports two-way binding.

11. Can signals replace RxJS completely?



No. Signals are useful for synchronous state; RxJS remains useful for asynchronous streams and complex event processing.

12. What is `untracked()`?



It reads a signal without registering that read as a dependency of the current reactive context.

13. How do you expose read-only state from a service?



Keep the `WritableSignal` private and expose its `asReadonly()` result.

14. What is the default signal equality comparison?



`Object.is()`. A custom equality function can be supplied when needed.

15. Does `effect()` execute synchronously for every `set()`?



19. Practical Interview Coding Challenge

Try answering these questions yourself.

Coding practice

4/4 revealed

1. Create a counter using signals.

 Copy

```
const count = signal(0);

increment() {
  this.count.update(x => x + 1);
}
```

Hide answer

2. Calculate total price using computed().

 Copy

```
price = signal(100);
quantity = signal(2);

total = computed(() =>
  this.price() * this.quantity()
);
```

Hide answer

3. Add an employee to a signal array.

 Copy

```
employees = signal<string[]>([]);

addEmployee(name: string) {
  this.employees.update(list => [
    ...list,
    name
  ]);
}
```

Hide answer

4. Create a read-only signal in a service.

 Copy

```
private _count = signal(0);

readonly count = this._count.asReadonly();
```

20. Final Interview Revision

Remember these key points

Quick revision before an Angular interview

0 of 12 revised

Reset

- ☐ `signal()` → Stores writable state
- ☐ `computed()` → Read-only derived state
- ☐ `effect()` → Runs side effects
- ☐ `linkedSignal()` → Writable derived state
- ☐ `input()` → Read-only component input
- ☐ `model()` → Two-way binding
- ☐ `set()` → Replace value
- ☐ `update()` → Calculate new value
- ☐ `asReadonly()` → Expose read-only state
- ☐ `untracked()` → Avoid dependency tracking
- ☐ `toSignal()` → Observable to Signal
- ☐ `toObservable()` → Signal to Observable

Most important interview question: Explain the difference between `signal()` , `computed()` , and `effect()` with a practical example. Understanding these three APIs provides the foundation for working with Angular Signals.